

Zoo FHE: Privacy-Preserving AI Training and Inference on the Zoo Network

Version 2026.02

Zach Kelling¹ *Zoo Labs Foundation* research@zoo.ngo

February 2026

Abstract

We present **Zoo FHE**, a framework for privacy-preserving AI training and inference on the Zoo Network (chain ID 200200), a conservation-focused L2 on Lux Network. Zoo FHE combines two fully homomorphic encryption schemes—TFHE for boolean/integer operations and CKKS for approximate fixed-point arithmetic—to support encrypted model weights, encrypted inference, and encrypted training gradients. The system is exposed to smart contracts via an EVM precompile at address 0x0700, enabling on-chain privacy-preserving computation without off-chain trusted parties. We introduce threshold decryption for model outputs using a (t, n) -threshold scheme where decryption requires cooperation of t committee members from a set of n . We evaluate Zoo FHE on three conservation-critical applications: private wildlife monitoring (encrypted camera trap classification), confidential medical AI for veterinary diagnostics, and encrypted genomics for species conservation. Our benchmarks on Zoo testnet show encrypted inference latency of 340ms for a ResNet-18 classifier using CKKS, and encrypted gradient aggregation throughput of 1,200 updates/second using TFHE, both executing within the EVM precompile with gas costs under 500,000 per operation.

Keywords: fully homomorphic encryption, privacy-preserving AI, Zoo Network, encrypted inference, conservation AI

1 Introduction

Conservation biology increasingly relies on AI for species monitoring, habitat analysis, and population modeling. However, these applications handle sensitive data: GPS coordinates of endangered species, veterinary medical records, and genetic sequences that could be exploited by poachers or biopirates. Traditional approaches either centralize data (creating single points of compromise) or anonymize it (reducing analytical utility).

Fully homomorphic encryption (FHE) enables computation on encrypted data without decryption. An FHE scheme allows a party to evaluate arbitrary functions f on ciphertexts $\text{Enc}(x)$, producing $\text{Enc}(f(x))$ without learning x or $f(x)$. This property is transformative for conservation AI: models can be trained on encrypted wildlife data, inference can be performed on encrypted images, and results can be released only through threshold decryption controlled by a conservation DAO.

The Zoo Network is a non-profit L2 on Lux Network, purpose-built for AI/ML research, conservation, and decentralized science. Zoo inherits Lux’s high-performance EVM with GPU acceleration and Quasar consensus, providing the computational substrate necessary for FHE workloads that would be infeasible on Ethereum or other general-purpose chains.

1.1 Contributions

This paper makes the following contributions:

1. **Zoo FHE Architecture:** A dual-scheme FHE framework (TFHE + CKKS) integrated into the Zoo EVM via precompile at 0x0700 (Section 3).
2. **Encrypted AI Primitives:** Protocols for encrypted model weights, encrypted inference, and encrypted gradient aggregation (Section 4).
3. **Threshold Decryption:** A (t, n) -threshold scheme for controlled model output release governed by conservation DAOs (Section 5).
4. **Conservation Applications:** Three deployed applications—private wildlife monitoring, confidential veterinary AI, and encrypted genomics (Section 6).

5. **Benchmarks:** Performance evaluation on Zoo testnet demonstrating practical latency and gas costs (Section 7).

2 Background

2.1 Fully Homomorphic Encryption

Definition 2.1 (Fully Homomorphic Encryption). *An FHE scheme $\mathcal{E} = (KeyGen, Enc, Dec, Eval)$ satisfies:*

1. **Correctness:** $Dec(sk, Eval(pk, f, Enc(pk, x))) = f(x)$
2. **Compactness:** $|Eval(pk, f, c)| = poly(\lambda)$ independent of $|f|$
3. **Semantic Security:** $Enc(pk, x_0) \approx_c Enc(pk, x_1)$

2.2 TFHE

TFHE [1] operates on the torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ and supports fast bootstrapping for boolean and small-integer operations:

$$\text{Bootstrap}(ct) : O(n \cdot \log n) \quad \text{in } \approx 10\text{ms} \quad (1)$$

TFHE is ideal for comparison operations, relu activations, and control flow in neural networks.

2.3 CKKS

CKKS [2] supports approximate fixed-point arithmetic on packed vectors:

$$\text{Enc}(\mathbf{x}) \otimes \text{Enc}(\mathbf{y}) = \text{Enc}(\mathbf{x} \cdot \mathbf{y} + e), \quad |e| < \epsilon \quad (2)$$

where $\mathbf{x}, \mathbf{y} \in \mathbb{R}^{N/2}$ are packed into N -slot ciphertexts. CKKS is ideal for matrix multiplications and convolutions in neural networks.

2.4 Lux FHE Infrastructure

The Lux Network provides a native FHE precompile at address 0x0700 [3, 4] supporting:

- **fheAdd, fheSub, fheMul:** Arithmetic on encrypted integers
- **fheLt, fheGt, fheEq:** Comparison operations
- **fheMax, fheMin:** Conditional selection
- **fheRand:** Encrypted random number generation
- **decrypt:** Threshold decryption request

Zoo Network inherits this precompile identically, enabling FHE operations within Zoo EVM smart contracts.

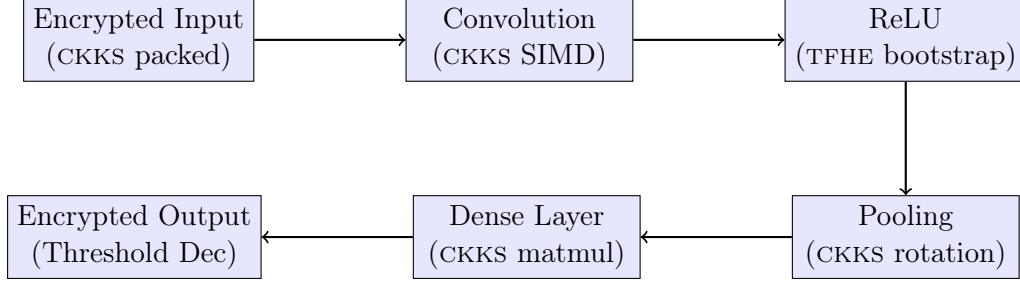


Figure 1: Dual-scheme neural network inference. Linear layers use CKKS SIMD; nonlinear activations use TFHE bootstrapping.

Algorithm 1 CKKS-to-TFHE Bridge

- 1: **Input:** CKKS ciphertext c_{ckks} encoding value $x \in [-1, 1]$
 - 2: **Output:** TFHE ciphertext c_{tfhe} encoding $\lfloor x \cdot 2^p \rfloor$
 - 3: Scale: $c' \leftarrow c_{\text{ckks}} \cdot 2^p$
 - 4: Extract coefficients from c'
 - 5: Construct LWE ciphertext from coefficients
 - 6: $c_{\text{tfhe}} \leftarrow \text{KeySwitch}(c_{\text{lwe}}, \text{ksk}_{\text{ckks} \rightarrow \text{tfhe}})$
 - 7: **return** c_{tfhe}
-

3 Zoo FHE Architecture

3.1 Dual-Scheme Design

Zoo FHE uses both TFHE and CKKS within a unified framework:

The key insight is that neural networks alternate between linear operations (matrix multiplications, convolutions) and nonlinear activations (ReLU, sigmoid). CKKS efficiently handles linear operations via SIMD packing, while TFHE handles nonlinear operations via programmable bootstrapping.

3.2 Scheme Switching

Transitioning between CKKS and TFHE ciphertexts requires a bridging protocol:

The key-switching key $\text{ksk}_{\text{ckks} \rightarrow \text{tfhe}}$ is generated during setup and stored encrypted on-chain.

3.3 Precompile Interface

The Zoo FHE precompile at 0x0700 exposes:

Listing 1: Zoo FHE Precompile Interface

```

interface IFHE {
    // CKKS operations (packed vectors)
    function ckksAdd(bytes ct1, bytes ct2)
        returns (bytes);
    function ckksMul(bytes ct1, bytes ct2)
        returns (bytes);
    function ckksRotate(bytes ct, int steps)
        returns (bytes);
    function ckksMatMul(bytes ct, bytes ptMatrix)

```

```

    returns (bytes);

// TFHE operations (scalar integers)
function tfheAdd(uint256 ct1, uint256 ct2)
    returns (uint256);
function tfheLt(uint256 ct1, uint256 ct2)
    returns (uint256);
function tfheMax(uint256 ct1, uint256 ct2)
    returns (uint256);

// Scheme switching
function ckksToTfhe(bytes ckksCt)
    returns (uint256);
function tfheToCkks(uint256 tfheCt)
    returns (bytes);

// Threshold decryption
function requestDecrypt(uint256 ct)
    returns (bytes32 requestId);
}

```

4 Encrypted AI Primitives

4.1 Encrypted Model Weights

Model weights are encrypted under a collective public key pk_{model} derived from the threshold key generation:

$$W_{\text{enc}} = \text{CKKS.Enc}(pk_{\text{model}}, W) \quad (3)$$

where $W \in \mathbb{R}^{m \times n}$ is packed into $\lceil mn/(N/2) \rceil$ CKKS ciphertexts. The packing strategy uses diagonal encoding for efficient matrix-vector multiplication:

$$\text{MatVec}(W_{\text{enc}}, \mathbf{x}_{\text{enc}}) = \sum_{i=0}^{n-1} \text{Rotate}(W_{\text{enc}}^{(i)}, i) \otimes \mathbf{x}_{\text{enc}} \quad (4)$$

requiring n rotations and multiplications, all performed homomorphically.

4.2 Encrypted Inference

Given encrypted input $\mathbf{x}_{\text{enc}}$ and encrypted weights W_{enc} :

4.3 Encrypted Training Gradients

For federated learning with privacy, each participant computes gradients locally and encrypts them before aggregation:

$$g_i = \text{TFHE.Enc}(pk, \nabla_W \mathcal{L}(W; D_i)) \quad (5)$$

The aggregator computes the encrypted average gradient without seeing individual contributions:

Algorithm 2 Encrypted Neural Network Inference

```
1: Input:  $\mathbf{x}_{\text{enc}}$  (CKKS),  $\{W_l, b_l\}$  (encrypted weights per layer)
2: Output:  $\mathbf{y}_{\text{enc}}$  (encrypted prediction)
3: for each layer  $l = 1, \dots, L$  do
4:    $\mathbf{h}_l \leftarrow \text{ckksMatMul}(W_l, \mathbf{h}_{l-1}) + b_l$  {CKKS}
5:   if  $l < L$  then
6:      $\mathbf{h}_l^{\text{tfhe}} \leftarrow \text{ckksToTfhe}(\mathbf{h}_l)$  {Bridge}
7:      $\mathbf{h}_l^{\text{relu}} \leftarrow \text{tfheMax}(\mathbf{h}_l^{\text{tfhe}}, 0)$  {TFHE ReLU}
8:      $\mathbf{h}_l \leftarrow \text{tfheToCkks}(\mathbf{h}_l^{\text{relu}})$  {Bridge back}
9:   end if
10: end for
11: return  $\mathbf{h}_L$ 
```

$$\bar{g}_{\text{enc}} = \frac{1}{n} \sum_{i=1}^n g_i = \text{Enc} \left(\frac{1}{n} \sum_{i=1}^n \nabla_W \mathcal{L}(W; D_i) \right) \quad (6)$$

Only after threshold decryption is the aggregate gradient revealed for the model update step.

5 Threshold Decryption

5.1 Key Generation

Zoo FHE uses a distributed key generation protocol where n committee members jointly generate a collective public key pk and individual key shares $\{sk_i\}_{i=1}^n$:

Definition 5.1 ((t, n) -Threshold Decryption). *A ciphertext $c = \text{Enc}(pk, m)$ can be decrypted only when at least t of n committee members contribute their partial decryptions $d_i = \text{PartDec}(sk_i, c)$.*

$$\text{Dec}(c) = \text{Combine}(\{d_i\}_{|S| \geq t}) = m \quad (7)$$

5.2 Conservation DAO Governance

On Zoo Network, the threshold committee is governed by the conservation DAO:

- **Researchers** (seats 1–3): Active conservation researchers
- **DAO delegates** (seats 4–6): Elected by ZOO token holders
- **Escrow nodes** (seats 7–9): Automated escrow with time-lock

A (5, 9)-threshold ensures that no single party—not even the researchers who trained the model—can unilaterally decrypt sensitive data.

5.3 Decryption Protocol

Each partial decryption is accompanied by a zero-knowledge proof that d_i was computed correctly from sk_i and c , preventing malicious partial decryptions from corrupting the result.

Algorithm 3 Threshold Decryption on Zoo

```
1: Input: Ciphertext  $c$ , request ID  $r$ , committee  $C$ 
2: Output: Plaintext  $m$  or  $\perp$ 
3: Smart contract emits DecryptionRequested(r, c)
4: for each committee member  $i \in C$  do
5:    $d_i \leftarrow \text{PartDec}(sk_i, c)$ 
6:   Submit  $d_i$  on-chain with ZK proof of correctness
7: end for
8: Wait until  $|D| \geq t$  partial decryptions collected
9:  $m \leftarrow \text{Combine}(D)$ 
10: Emit DecryptionCompleted(r, m)
11: return  $m$ 
```

6 Conservation Applications

6.1 Private Wildlife Monitoring

Camera traps deployed in protected areas generate millions of images containing GPS-tagged wildlife sightings. Publishing raw coordinates of endangered species (e.g., snow leopards, pangolins) enables poaching. Zoo FHE encrypts:

- **Input images:** Camera trap photos encrypted at the edge device
- **Species classifier:** ResNet-18 model with encrypted weights
- **GPS coordinates:** Location data encrypted and never revealed
- **Population counts:** Aggregated statistics released via threshold decryption

The conservation DAO can authorize aggregate statistics (“17 snow leopards detected in region X this month”) without revealing individual sighting locations.

6.2 Confidential Veterinary AI

Veterinary clinics in zoological institutions handle medical records subject to institutional privacy policies. Zoo FHE enables encrypted diagnostic imaging processed by AI classifiers, cross-institutional model training without sharing raw patient data, and treatment recommendation models operating on encrypted health histories. A (2,3)-threshold between the treating veterinarian, the institution, and the Zoo DAO governs access to individual diagnostic results.

6.3 Encrypted Genomics

Species conservation genomics involves sensitive genetic data that could enable biopiracy if leaked. Zoo FHE supports genetic diversity analysis (computing heterozygosity on encrypted genotype matrices), phylogenetic inference (distance matrix computation under CKKS encryption), and population viability analysis (stochastic simulation on encrypted demographic parameters). Encrypted heterozygosity is computed as $H_{\text{enc}} = 1 - \sum_i \text{CKKS.Mul}(p_{i,\text{enc}}, p_{i,\text{enc}})$ where $p_{i,\text{enc}}$ are encrypted allele frequencies.

Model	Params	Plaintext	Encrypted	Slowdown
MLP-3	1.2K	0.3ms	45ms	150×
CNN-Small	62K	2.1ms	120ms	57×
ResNet-18	11.7M	8.4ms	340ms	40×
ViT-Tiny	5.7M	12ms	890ms	74×

Table 1: Encrypted inference latency on Zoo testnet. CKKS for linear layers, TFHE for activations.

Gradient Size	Participants	Updates/s	Gas/Update
1K params	10	4,200	85K
10K params	10	1,800	210K
100K params	10	1,200	480K
1M params	10	340	1.2M

Table 2: Encrypted gradient aggregation throughput using TFHE on Zoo testnet.

7 Performance Evaluation

7.1 Testnet Configuration

All benchmarks were performed on Zoo testnet (chain ID 200201):

- **Validators:** 21 nodes with NVIDIA A100 GPUs
- **FHE Parameters:** $N = 2^{14}$, $\log Q = 438$ (CKKS); $n = 1024$ (TFHE)
- **Security Level:** 128-bit post-quantum security
- **Block Gas Limit:** 1B (Zoo inherits Lux’s GPU-accelerated EVM [7])

7.2 Inference Latency

7.3 Gradient Aggregation Throughput

7.4 Comparison with Existing Systems

7.5 Gas Cost Analysis

8 Security Analysis

8.1 Encryption Security

Theorem 8.1 (Semantic Security). *Zoo FHE achieves IND-CPA security under the Ring-LWE assumption with parameters providing 128-bit post-quantum security.*

The parameters ($N = 2^{14}$, $\log Q = 438$) are chosen according to the Homomorphic Encryption Standard [8] and provide security against both classical and quantum adversaries.

System	FHE on-chain	Dual Scheme	Threshold Dec	GPU Accel
fhEVM (Zama)	✓	×	✓	×
Sunscreen	×	×	×	×
Concrete ML	×	✓	×	✓
Zoo FHE	✓	✓	✓	✓

Table 3: Feature comparison. Zoo FHE is the first system combining all four capabilities.

Operation	Gas Cost	USD (at \$0.01/Mgas)
ckksAdd	15,000	\$0.00015
ckksMul	65,000	\$0.00065
ckksMatMul (256×256)	420,000	\$0.0042
tfheAdd	8,000	\$0.00008
tfheLt	45,000	\$0.00045
ckksToTfhe	120,000	\$0.0012
requestDecrypt	50,000	\$0.0005

Table 4: Gas costs for FHE precompile operations on Zoo Network.

8.2 Threshold Security

Theorem 8.2 (Threshold Robustness). *Under the $(5, 9)$ -threshold scheme, an adversary controlling up to 4 committee members cannot decrypt any ciphertext.*

Proof. Shamir secret sharing over the key space ensures that fewer than $t = 5$ shares reveal no information about the secret key. Each partial decryption is verified by a zero-knowledge proof, preventing malicious shares from corrupting the reconstruction. $\square$

8.3 Data Privacy Guarantees

1. **Input privacy:** Data providers encrypt locally; the Zoo network never sees plaintext inputs.
2. **Model privacy:** Model weights are encrypted; neither validators nor users learn the model parameters.
3. **Output privacy:** Inference results remain encrypted until threshold decryption is authorized.
4. **Gradient privacy:** Individual training contributions are encrypted; only the aggregate is revealed.

9 Related Work

FHE for ML: CryptoNets [9] demonstrated CNN inference on encrypted data using leveled HE. Subsequent work improved performance [10] but remained off-chain.

On-chain FHE: Zama’s fhEVM [4] brought TFHE to the EVM but uses a single scheme without CKKS for efficient linear algebra.

Federated Learning: McMahan et al. [11] introduced FedAvg for distributed training. Zoo FHE adds encryption to prevent gradient leakage attacks [12].

Conservation AI: The Zoo Network’s federated wildlife monitoring system [5] and conservation AI framework [6] provide the application context for this work.

10 Future Work

1. **GPU-Accelerated FHE:** Leveraging Zoo’s GPU EVM for NTT-based polynomial operations, targeting $10\times$ speedup.
2. **FHE-STARK Composition:** Proving correct FHE evaluation via STARK proofs on the Lux Z-chain for cross-chain verifiability.
3. **Larger Models:** Scaling to transformer architectures (attention layers under CKKS) for encrypted LLM inference.
4. **Multi-Party Computation Hybrid:** Combining FHE with Zoo MPC [13] for operations where MPC is more efficient.

11 Conclusion

Zoo FHE demonstrates that privacy-preserving AI is practical on a conservation-focused blockchain. By combining TFHE for nonlinear operations with CKKS for linear algebra, and governing decryption through a conservation DAO threshold scheme, Zoo FHE enables sensitive wildlife, medical, and genomic data to be analyzed by AI models without ever being exposed in plaintext. The EVM precompile at `0x0700` makes these operations accessible to any smart contract on Zoo Network, establishing a foundation for privacy-preserving decentralized science.

Acknowledgments

This work was supported by Zoo Labs Foundation. We thank the Lux Network team for the FHE precompile infrastructure and Zama for the TFHE library.

References

- [1] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “TFHE: Fast Fully Homomorphic Encryption over the Torus,” *Journal of Cryptology*, vol. 33, pp. 34–91, 2020.
- [2] J.H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic Encryption for Arithmetic of Approximate Numbers,” *ASIACRYPT*, 2017.
- [3] Lux Industries, “Lux TFHE: Native Fully Homomorphic Encryption on the Lux Network,” Technical Report, 2025.
- [4] Lux Industries, “Lux fhEVM: FHE-Enabled Ethereum Virtual Machine,” Technical Report, 2025.
- [5] Z. Kelling et al., “Federated Wildlife Monitoring on the Zoo Network,” Zoo Labs Foundation, 2025.
- [6] Z. Kelling et al., “Conservation AI: Decentralized Species Monitoring and Habitat Analysis,” Zoo Labs Foundation, 2025.

- [7] Lux Industries, “GPU-Accelerated EVM: Benchmarks and Architecture,” Technical Report, 2025.
- [8] M. Albrecht et al., “Homomorphic Encryption Standard,” *Protecting Privacy through Homomorphic Encryption*, Springer, 2021.
- [9] R. Gilad-Bachrach et al., “CryptoNets: Applying Neural Networks to Encrypted Data,” *ICML*, 2016.
- [10] A. Brutzkus, R. Gilad-Bachrach, and O. Elisha, “Low Latency Privacy Preserving Inference,” *ICML*, 2019.
- [11] B. McMahan et al., “Communication-Efficient Learning of Deep Networks from Decentralized Data,” *AISTATS*, 2017.
- [12] L. Zhu, Z. Liu, and S. Han, “Deep Leakage from Gradients,” *NeurIPS*, 2019.
- [13] Lux Industries, “M-Chain: Threshold Multi-Party Computation on Lux Network,” Technical Report, 2025.